



Puesta a punto del estudiante

Cuenta de GitHub, control de versiones y entorno de trabajo

Modelación y Simulación Computacional · Maestría en Ingeniería · Universidad de Sucre · Periodo 2026-2 · Prof. Daniel D. Otero Meza

1. Qué hay que tener listo y cuándo

La asignatura evalúa el repositorio y no solamente el resultado, porque el plan declara el trabajo colaborativo con control de versiones como estrategia metodológica y porque el proyecto integrador se entrega como un repositorio que otra persona debe poder ejecutar. Por eso las primeras dos horas del curso se dedican a dejar tres cosas funcionando, que son la cuenta, el control de versiones y el entorno de cálculo.

La [Tabla 1](#) indica qué se necesita en cada momento. Lo de la primera fila se hace antes de llegar al aula y toma menos de veinte minutos. Lo demás se hace acompañado en la sesión del viernes.

Tabla 1: Lo que debe estar listo en cada momento de la primera semana.

Momento	Qué debe estar hecho	Dónde se explica
Antes de la primera sesión	Cuenta de GitHub creada y nombre de usuario enviado al docente por el canal del curso	Secciones 2 y 3
Primera franja del viernes	Git instalado y configurado, e invitación a la organización aceptada	Secciones 4 y 5
Segunda franja del viernes	Entorno de cálculo funcionando, sea local o en la nube	Sección 6
Cierre del viernes	Cuaderno de puesta a punto ejecutado completo y sin errores	Sección 7

Quien llegue sin nada de esto no queda excluido de la sesión, pero pierde la franja de laboratorio tratando de instalar en lugar de modelar. Conviene hacer la sección 2 hoy mismo, aunque sea desde el teléfono, porque la invitación a la organización necesita que la cuenta exista.

2. La cuenta de GitHub

GitHub es el lugar donde vive la copia publicada de cada repositorio y donde el docente lee su trabajo. La cuenta es gratuita y se crea en <https://github.com>.

1. Regístrese con el correo institucional, porque es el que permite pedir después los beneficios académicos y el que identifica su trabajo.
2. Escoja un nombre de usuario que pueda decir en voz alta y que sirva fuera del curso, porque es una dirección pública y permanente. Algo como `dotero-unisuc` envejece mejor que un apodo.
3. Verifique el correo desde el mensaje que llega en seguida.
4. Active la verificación en dos pasos cuando la plataforma la pida, con una aplicación de códigos en el teléfono. Guarde los códigos de recuperación en un lugar distinto del teléfono.



Lo primero que hay que entregar

Escriba su nombre completo y su nombre de usuario de GitHub como comentario en el asunto titulado *Registro de nombres de usuario* del repositorio del material, o por el canal de mensajería del curso si todavía no tiene cuenta. Sin ese dato el docente no puede invitarlo a la organización y usted no podrá entregar ni el Taller 1 ni la primera entrega del proyecto.

3. Beneficios académicos, que son opcionales

GitHub ofrece un paquete de beneficios para estudiantes que se solicita en <https://education.github.com>, con el carné y el correo institucional. La aprobación puede tardar varios días y no hace falta para nada de la asignatura, de modo que solicítelo si le interesa, pero no espere a que llegue para hacer lo demás.

4. Instalar y configurar Git

Git es el programa que registra la historia de los cambios en su computador. GitHub es otra cosa, es el servicio donde esa historia se publica. Se instala una sola vez.

En Windows se descarga de <https://git-scm.com/download/win> y se instala con las opciones por omisión, salvo dos. En la pantalla de la rama inicial hay que escoger *main*, y en la de la línea de órdenes hay que dejar que el instalador agregue Git al PATH. En macOS basta con escribir *git* en la Terminal y aceptar la instalación de las herramientas de desarrollo que ofrece el sistema. En Linux se instala con el gestor de paquetes de la distribución.

Después, en PowerShell si usa Windows o en la Terminal en los demás casos, se escriben estas cuatro órdenes una sola vez. El correo debe ser el mismo de la cuenta de GitHub, porque es lo que hace que sus envíos aparezcan a su nombre.

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@unisucra.edu.co"
git config --global init.defaultBranch main
git config --global core.quotepath false
```

Para comprobar que quedó bien se usa `git config -global -list`, que debe mostrar su nombre y su correo. La primera vez que publique algo, Git abrirá el navegador para que autorice el acceso a su cuenta, y desde entonces no volverá a pedirlo. La contraseña de la cuenta no sirve para publicar.

5. La organización del curso y sus tres repositorios

Todo el curso vive en la organización *msc-unisucra* de GitHub, con tres repositorios que cumplen funciones distintas, según la [Tabla 2](#). Usted recibirá una invitación por correo para unirse, y hay que aceptarla antes de la primera franja del viernes.

Tabla 2: Los tres repositorios del curso y lo que usted hace en cada uno.

Repositorio	Qué contiene	Qué hace usted allí
<i>msc2026-material</i>	Los cuadernos del curso y sus datos	Lo consulta y lo abre en Colab, no escribe en él
<i>msc2026-talleres</i>	Los cuatro cuadernos de taller	Trabaja en la rama de su pareja y entrega con una solicitud de incorporación
<i>msc2026-plantilla-proyecto</i>	La plantilla del proyecto integrador	La usa una vez para crear su propio repositorio de proyecto



Su repositorio de proyecto se crea en la franja del viernes dedicada al lanzamiento, a partir de la plantilla y con el botón que GitHub ofrece para eso. El nombre es **proyecto-apellido** y la visibilidad privada, y se crea dentro de la organización y no en su cuenta personal, para que el docente pueda leerlo sin más trámites.

6. El entorno de cálculo

Hay dos caminos y los dos sirven. El primero es instalar el entorno en su computador, que es lo recomendable porque el proyecto va a crecer y porque el informe final debe reconstruirse con una sola orden. El segundo es trabajar en Google Colab, que no exige instalar nada y que es la salida razonable si su equipo tiene poca memoria o si el primer camino falla.

Para el camino local se instala Miniconda desde <https://www.anaconda.com/download/success>, se descarga el archivo `environment.yml` del repositorio del material y se ejecutan las órdenes siguientes en la carpeta donde quedó ese archivo. En Windows hay que escribir la primera una sola vez y después cerrar y volver a abrir PowerShell.

```
conda init powershell
conda env create -f environment.yml
conda activate msc2026
jupyter lab
```

El entorno se llama `msc2026` y trae Python 3.11 con NumPy, SciPy, pandas, Matplotlib, SymPy, JupyterLab y pytest en las versiones exactas que usan los cuadernos. Esa exactitud es lo que permite que los números del libro coincidan con los suyos.

Para el camino en la nube basta con abrir cualquier cuaderno desde la insignia azul de Colab que aparece en su encabezado, con la sesión de Google iniciada. Los cuadernos detectan que están en Colab e instalan por su cuenta lo que falte. Tenga presente que Colab borra los archivos al cerrar la sesión, de modo que lo que produzca hay que descargarlo o enviarlo a su repositorio antes de salir.

7. La comprobación que decide si quedó bien

Abra el cuaderno `U0_00_puesta_a_punto.ipynb` del repositorio del material, reinicie el núcleo y ejecútelo de principio a fin. Ese cuaderno verifica las versiones de las cinco bibliotecas, fija la semilla 20 262 del curso, demuestra que dos corridas con esa semilla coinciden dígito a dígito, dibuja una figura con la paleta del libro y comprueba que Git responde. Si termina sin ninguna aserción fallida, usted está listo.

Si alguna versión queda por debajo de la mínima, actualícela con `pip install -U nombre` y vuelva a ejecutar el cuaderno completo. Si la lectura de datos falla, confirme desde qué carpeta abrió el cuaderno, porque la función que localiza los archivos prueba datos, `../datos` y `../../datos` en ese orden.

8. Las nueve órdenes que usa el curso

Git tiene centenares de órdenes y el curso usa nueve, que son las de la [Tabla 3](#). Con ellas se hace todo lo que exigen los talleres y el proyecto, y no hace falta aprender el resto para esta asignatura.

Un día de trabajo tiene esta forma. Se trae lo que haya de nuevo, se hace el trabajo, se revisa qué cambió, se registra por partes con mensajes que expliquen la intención y se publica. Registrar por partes es lo que hace legible un historial, porque un envío que toca once archivos por once razones distintas no se puede revisar ni revertir.

```
git clone https://github.com/msc-unisucree/proyecto-apellido.git
cd proyecto-apellido
```

**Tabla 3:** Las nueve órdenes de Git que el curso exige, con su propósito.

Orden	Para qué sirve
<code>git status</code>	Ver qué cambió desde el último registro
<code>git add ruta/al/archivo</code>	Escoger qué entra al registro, archivo por archivo
<code>git commit -m "mensaje"</code>	Registrar el cambio con un mensaje en presente
<code>git log --oneline -5</code>	Revisar el historial reciente
<code>git switch -c nombre-de-rama</code>	Abrir una rama para una prueba
<code>git switch main</code>	Volver a la rama principal
<code>git merge nombre-de-rama</code>	Incorporar la prueba que funcionó
<code>git pull</code>	Traer lo que publicaron los demás
<code>git push</code>	Publicar el trabajo propio

```
git pull
git status
git add src/modelo.py
git commit -m "Ajusta el coeficiente de reaireación al rango de estiaje"
git push
```

El mensaje del ejemplo dice qué se hizo y sobre qué. Un mensaje como cambios o avance no permite entender la historia tres semanas después, y la rúbrica del proyecto lo penaliza.

9. Cómo se entrega

Los talleres se entregan en la rama de la pareja dentro del repositorio de talleres, con el nombre del taller y los dos apellidos, por ejemplo `taller1-otero-villadiego`, y se cierran abriendo una solicitud de incorporación contra la rama principal. Esa solicitud es la entrega y no hay que enviar nada por otro medio, porque la fecha que cuenta es la del último envío publicado antes del plazo. El docente anuncia en la primera sesión el plazo de cada instrumento, y no es el mismo para los talleres que para las entregas del proyecto.

Las cuatro entregas del proyecto se marcan con una etiqueta en su propio repositorio, que congela el estado entregado para que la calificación recaiga sobre algo concreto y no sobre lo que usted siga cambiando después. La etiqueta se crea y se publica con dos órdenes.

```
git tag -a E1 -m "Entrega 1, propuesta de tema y modelo conceptual"
git push origin E1
```

La entrega es esa etiqueta publicada, cuya dirección termina en `msc-unisucere/proyecto-apellido/tree/E1`. En las entregas que incluyen informe, el documento exportado va dentro del repositorio y no por otro medio. La fuente de verdad es siempre el repositorio.

Por qué se exige el historial

La guía del proyecto pide que el historial muestre trabajo distribuido en las cuatro semanas y no un único envío final. La razón no es administrativa. Un historial fechado es la única evidencia de que los indicadores y sus umbrales se fijaron antes de conocer los resultados, que es lo que distingue un análisis de una racionalización.

10. Qué se versiona y qué no

La regla cabe en una línea. Se versiona todo lo que un ser humano escribió y no puede reconstruirse, y no se versiona nada que una orden pueda regenerar. Los datos crudos entran y no se tocan nunca más, el código entra, los cuadernos entran, y quedan fuera los datos procesados, las figuras, las tablas de



resultados y los archivos temporales, porque el código los vuelve a producir.

Los cuadernos merecen un cuidado adicional. Un archivo con extensión `.ipynb` guarda las salidas junto con el código, de modo que dos personas que ejecutan el mismo cuaderno producen archivos distintos y un conflicto ilegible. Reinicie el núcleo y borre las salidas antes de registrar un cuaderno de trabajo. La única excepción es el cuaderno del informe final, que se entrega ejecutado porque sus salidas son el resultado.

11. Problemas frecuentes

La [Tabla 4](#) recoge lo que suele pasar la primera semana. Ante un mensaje que no se entiende, la regla más útil es leerlo y no repetir la orden, porque Git casi siempre dice exactamente qué hacer.

Tabla 4: Problemas frecuentes de la primera semana y su solución.

Situación	Salida
El envío se rechaza porque el remoto tiene cambios	<code>git pull</code> y luego <code>git push</code> , resolviendo el conflicto si aparece
Aparecen marcas con signos de menor y de mayor dentro de un archivo	Es un conflicto. Abra el archivo, deje el texto que debe quedar, borre las tres líneas de marca y registre el archivo
El mensaje del último envío quedó mal escrito	<code>git commit --amend -m "mensaje correcto"</code> , siempre que no se haya publicado
Se modificó un archivo y se quiere descartar el cambio	<code>git restore ruta/al/archivo</code>
<code>conda activate</code> no funciona en Windows	Ejecute <code>conda init powershell</code> una vez y vuelva a abrir la ventana
El entorno no se crea por falta de espacio o de memoria	Trabaje en Colab esta semana y resuelva la instalación como trabajo independiente

Lo que no conviene hacer es guardar un repositorio dentro de una carpeta sincronizada con un servicio de nube, porque la carpeta oculta que Git usa como base de datos se corrompe cuando dos sistemas la escriben a destiempo. Cree una carpeta propia para sus repositorios y trabaje allí.